

SETUP AND ACCESS

Configure auto mode

Tell the auto mode classifier which repos, buckets, and domains your organization trusts. Set environment context, override the default block and allow rules, and inspect your effective config with the auto-mode CLI subcommands.

Auto mode lets Claude Code run without routine permission prompts by routing tool calls through a classifier that blocks anything irreversible, destructive, or aimed outside your environment. Deny and explicit ask rules are evaluated before the classifier and still block or prompt. Use the `autoMode` settings block to tell that classifier which repos, buckets, and domains your organization trusts, so it stops blocking routine internal operations.

❗ Auto mode is available to all users on the Anthropic API. On Amazon Bedrock, Google Cloud Vertex AI, and Microsoft Foundry, you must first [set CLAUDE_CODE_ENABLE_AUTO_MODE](#). If Claude Code reports auto mode as unavailable for your account, check the [full requirements](#), which also cover the supported models and Owner enablement on Team and Enterprise plans.

By default, the classifier trusts only the working directory and the current repo's configured remotes. Actions like pushing to your company's source-control org or writing to a team cloud bucket are blocked until you add them to `autoMode.environment`.

For how to enable auto mode and what it blocks by default, see [Permission modes](#). This page is the configuration reference.

This page covers how to:

- [Choose where to set rules](#) across CLAUDE.md, user settings, and managed settings
- [Define trusted infrastructure](#) with `autoMode.environment`
- [Override the block and allow rules](#) when the defaults don't fit your pipeline
- [Inspect your effective config](#) with the `claude auto-mode` subcommands
- [Review denials](#) so you know what to add next

Where the classifier reads configuration

The classifier reads the same **CLAUDE.md** content Claude itself loads, so an instruction like “never force push” in your project’s CLAUDE.md steers both Claude and the classifier at the same time. Start there for project conventions and behavioral rules.

For rules that apply across projects, such as trusted infrastructure or organization-wide deny rules, use the `autoMode` settings block. The classifier reads `autoMode` from the following scopes:

Scope	File	Use for
One developer	<code>~/.claude/settings.json</code>	Personal trusted infrastructure
One project, one developer	<code>.claude/settings.local.json</code>	Per-project trusted buckets or services
Organization-wide	<u>Managed settings</u>	Trusted infrastructure distributed to all developers
<code>--settings</code> flag or Agent SDK	Inline JSON	Per-invocation overrides for automation

The classifier doesn’t read `autoMode` from shared project settings in `.claude/settings.json`, so a checked-in repo can’t inject its own allow rules.

Entries from each scope are combined. A developer can extend `environment`, `allow`, `soft_deny`, and `hard_deny` with personal entries but can’t remove entries that managed settings provide. Because allow rules act as exceptions to soft block rules inside the classifier, a developer-added `allow` entry can override an organization `soft_deny` entry: the combination is additive, not a hard policy boundary.

❗ The classifier is a second gate that runs after the [permissions system](#). For actions that must never run regardless of user intent or classifier configuration, use `permissions.deny` in managed settings, which blocks the action before the classifier is consulted and can’t be overridden.

Define trusted infrastructure

For most organizations, `autoMode.environment` is the only field you need to set. It tells the classifier which repos, buckets, and domains are trusted: the classifier uses it to decide what “external” means, so any destination not listed is a potential exfiltration target.

The default environment list trusts the working repo and its configured remotes. To add your own

entries alongside that default, include the literal string `"$defaults"` in the array. The default entries are spliced in at that position, so your custom entries can go before or after them.

```
{
  "autoMode": {
    "environment": [
      "$defaults",
      "Source control: github.example.com/acme-corp and all repos
under it",
      "Trusted cloud buckets: s3://acme-build-artifacts,
gs://acme-ml-datasets",
      "Trusted internal domains: *.corp.example.com,
api.internal.example.com",
      "Key internal services: Jenkins at ci.example.com,
Artifactory at artifacts.example.com"
    ]
  }
}
```

Entries are prose, not regex or tool patterns. The classifier reads them as natural-language rules. Write them the way you would describe your infrastructure to a new engineer. A thorough environment section covers:

- **Organization:** your company name and what Claude Code is primarily used for, like software development, infrastructure automation, or data engineering
- **Source control:** every GitHub, GitLab, or Bitbucket org your developers push to
- **Cloud providers and trusted buckets:** bucket names or prefixes that Claude should be able to read from and write to
- **Trusted internal domains:** hostnames for APIs, dashboards, and services inside your network, like `*.internal.example.com`
- **Key internal services:** CI, artifact registries, internal package indexes, incident tooling
- **Additional context:** regulated-industry constraints, multi-tenant infrastructure, or compliance requirements that affect what the classifier should treat as risky

A useful starting template: fill in the bracketed fields and remove any lines that don't apply.

```

{
  "autoMode": {
    "environment": [
      "$defaults",
      "Organization: {COMPANY_NAME}. Primary use:
{PRIMARY_USE_CASE, e.g. software development, infrastructure
automation}",
      "Source control: {SOURCE_CONTROL, e.g. GitHub org
github.example.com/acme-corp}",
      "Cloud provider(s): {CLOUD_PROVIDERS, e.g. AWS, GCP,
Azure}",
      "Trusted cloud buckets: {TRUSTED_BUCKETS, e.g. s3://acme-
builds, gs://acme-datasets}",
      "Trusted internal domains: {TRUSTED_DOMAINS, e.g.
*.internal.example.com, api.example.com}",
      "Key internal services: {SERVICES, e.g. Jenkins at
ci.example.com, Artifactory at artifacts.example.com}",
      "Additional context: {EXTRA, e.g. regulated industry,
multi-tenant infrastructure, compliance requirements}"
    ]
  }
}

```

The more specific context you give, the better the classifier can distinguish routine internal operations from exfiltration attempts.

You don't need to fill everything in at once. A reasonable rollout: start with the defaults and add your source control org and key internal services, which resolves the most common false positives like pushing to your own repos. Add trusted domains and cloud buckets next. Fill the rest as blocks come up.

Override the block and allow rules

Three additional fields let you replace the classifier's built-in rule lists:

- `autoMode.hard_deny` : unconditional security boundaries
- `autoMode.soft_deny` : destructive actions that user intent can clear
- `autoMode.allow` : exceptions to soft block rules

Each is an array of prose descriptions, read as natural-language rules. For tool-pattern-based hard

blocks that run before the classifier, use `permissions.deny` .

Inside the classifier, precedence works in four tiers:

- `hard_deny` rules block unconditionally. User intent and `allow` exceptions don't apply.
- `soft_deny` rules block next. User intent and `allow` exceptions can override these.
- `allow` rules then override matching `soft_deny` rules as exceptions.
- Explicit user intent overrides the remaining soft blocks: if the user's message directly and specifically describes the exact action Claude is about to take, the classifier allows it even when a `soft_deny` rule matches.

General requests don't count as explicit intent. Asking Claude to "clean up the repo" doesn't authorize force-pushing, but asking Claude to "force-push this branch" does.

To loosen, add to `allow` when the classifier repeatedly flags a routine pattern the default exceptions don't cover. To tighten, add to `soft_deny` for destructive risks specific to your environment that the defaults miss, or to `hard_deny` for security boundaries that must never be crossed.

To keep the built-in rules while adding your own, include the literal string `"$defaults"` in the array. The default rules are spliced in at that position, so your custom rules can go before or after them, and you continue to inherit updates as the built-in list changes across releases.

The following example keeps the defaults in all four lists and adds organization-specific rules to each.

```

{
  "autoMode": {
    "environment": [
      "$defaults",
      "Source control: github.example.com/acme-corp and all repos
under it"
    ],
    "allow": [
      "$defaults",
      "Deploying to the staging namespace is allowed: staging is
isolated from production and resets nightly",
      "Writing to s3://acme-scratch/ is allowed: ephemeral bucket
with a 7-day lifecycle policy"
    ],
    "soft_deny": [
      "$defaults",
      "Never run database migrations outside the migrations CLI,
even against dev databases",
      "Never modify files under infra/terraform/prod/: production
infrastructure changes go through the review workflow"
    ],
    "hard_deny": [
      "$defaults",
      "Never send repository contents to third-party code-review
APIs"
    ]
  }
}

```

⚠ Setting any of `environment`, `allow`, `soft_deny`, or `hard_deny` without `"$defaults"` replaces the entire default list for that section. A `soft_deny` array without `"$defaults"` discards every built-in soft block rule, including force push, `curl | bash`, and production deploys. A `hard_deny` array without `"$defaults"` discards the built-in data exfiltration and auto-mode bypass rules.

Each section is evaluated independently, so setting `environment` alone leaves the default `allow`, `soft_deny`, and `hard_deny` lists intact.

Only omit `"$defaults"` when you intend to take full ownership of the list. To do that safely, run `claude auto-mode defaults` to print the built-in rules, copy them into your settings file, then review each rule against your own pipeline and risk tolerance.

Inspect the defaults and your effective config

Three CLI subcommands help you inspect and validate your configuration.

Print the built-in `environment` , `allow` , `soft_deny` , and `hard_deny` rules as JSON:

```
claude auto-mode defaults
```

Print what the classifier actually uses as JSON, with your settings applied where set and defaults otherwise:

```
claude auto-mode config
```

Get AI feedback on your custom `allow` , `soft_deny` , and `hard_deny` rules:

```
claude auto-mode critique
```

Run `claude auto-mode config` after saving your settings to confirm the effective rules are what you expect, with `"$defaults"` expanded in place. If you've written custom rules, `claude auto-mode critique` reviews them and flags entries that are ambiguous, redundant, or likely to cause false positives.

If you need to remove or rewrite a built-in rule rather than add alongside it, save the output of `claude auto-mode defaults` to a file, edit the lists, and paste the result into your settings file in place of `"$defaults"` .

Review denials

When auto mode denies a tool call, the denial is recorded in `/permissions` under the Recently denied tab. Press `r` on a denied action to mark it for retry: when you exit the dialog, Claude Code sends a message telling the model it may retry that tool call and resumes the conversation.

Repeated denials for the same destination usually mean the classifier is missing context. Add that destination to `autoMode.environment` , then run `claude auto-mode config` to confirm it took effect.

To react to denials programmatically, use the `PermissionDenied` hook.

See also

- **Permission modes**: what auto mode is, what it blocks by default, and how to enable it
- **Managed settings**: deploy `autoMode` configuration across your organization
- **Permissions**: allow, ask, and deny rules that apply before the classifier runs
- **Settings**: the full settings reference, including the `autoMode` key